



BITS Pilani
Pilani Campus

API-driven Cloud Native Solutions

Dr. Shreyas Rao
Associate Prof. (Off Campus), CSIS, BITS-Pilani



CC ZG506 / SE ZG506

Lecture No. 7

Agenda

- Machine Learning
- Generations
- Machine Learning Pipeline
- MLOps
- Tools: MLFlow and AWS SageMaker
- API-driven ML pipelines
 - Model Development and Training
 - Model Deployment
 - Monitoring
 - Scalability



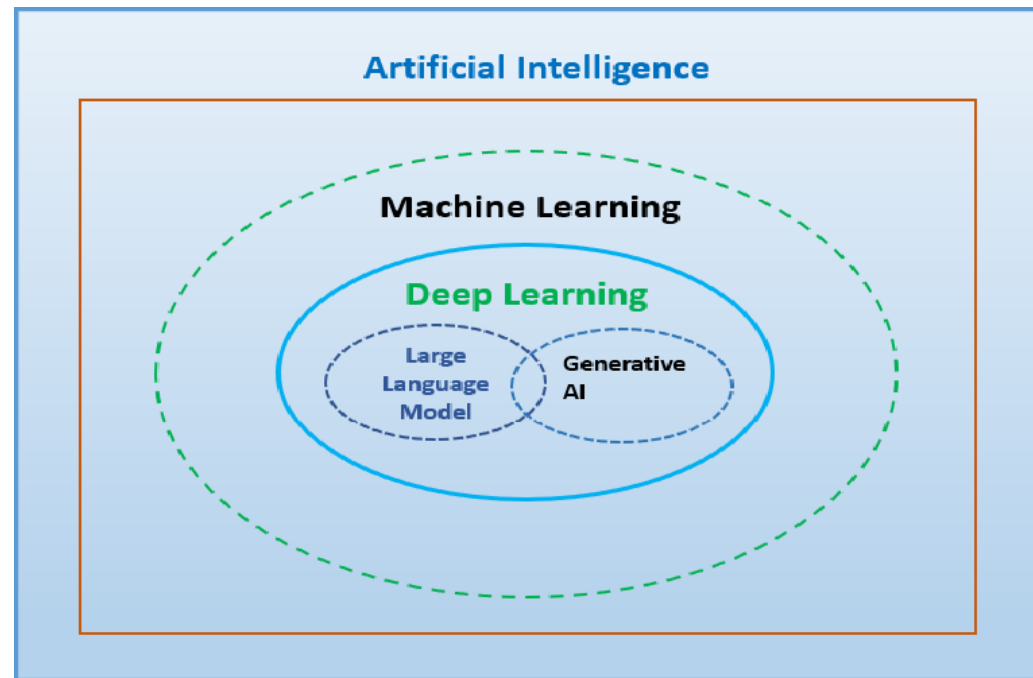
BITS Pilani
Pilani Campus



Machine Learning

Machine Learning

Machine learning (ML) is a branch of artificial intelligence (AI) and computer science that focuses on using data and algorithms to enable AI to imitate the way that humans learn, gradually improving its accuracy.



Machine Learning Complexity



Data

Schema

Sampling over Time

Volume



Model

Algorithms

More Training

Experiments



Code

Business Needs

Bug Fixes

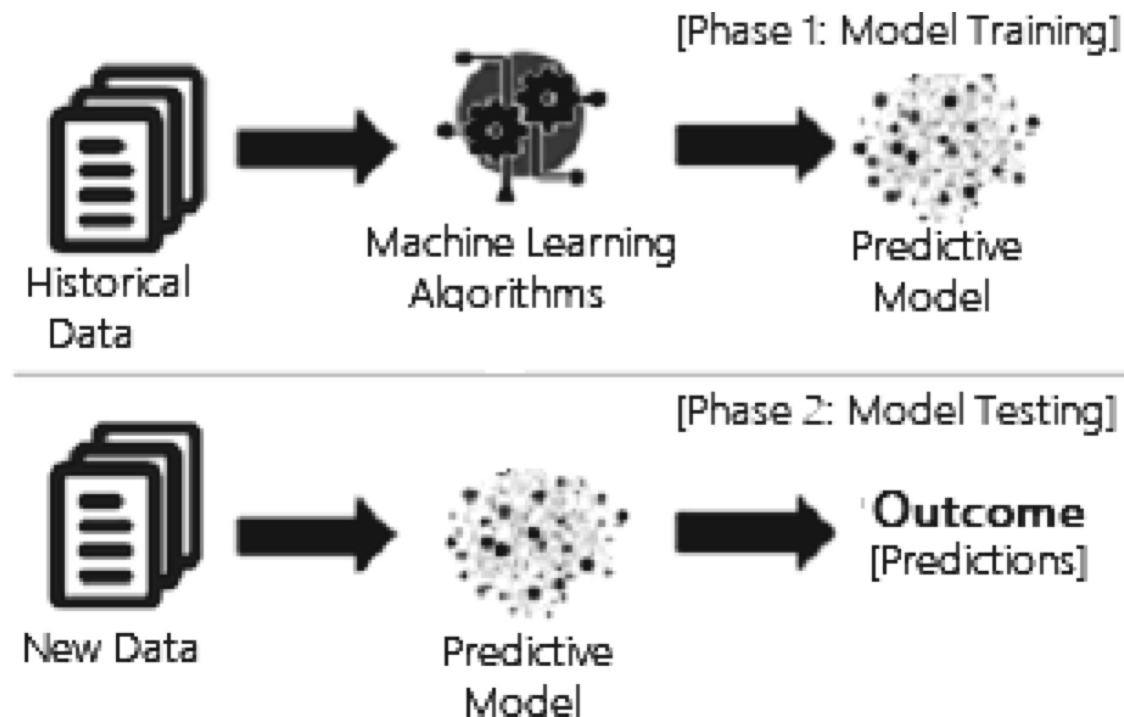
Configuration

Number of changing variables - 3 (Data, Model and Code), whereas in pure software development, only changing variable is Code

<https://martinfowler.com/articles/cd4ml.html>

Model in ML

A model in Machine Learning is the trained algorithm used to make predictions or decisions, and it forms the core of a machine learning system that is developed, deployed, monitored, and maintained as part of an operational pipeline.





Model Workflow

1. **Model Development:** Building and training the model using training data. This involves selecting an algorithm, tuning hyper parameters, and evaluating performance.
2. **Model Versioning:** Keeping track of different versions of a model, which may be developed over time to improve accuracy or adapt to new data.
3. **Model Deployment:** Taking the trained model and deploying it into a production environment where it can make predictions on live data.
4. **Model Monitoring:** Continuously monitoring the model's performance in production to ensure that it is accurate and effective. This may involve detecting data drift, model degradation, or biases.
5. **Model Retraining:** Updating or retraining the model periodically with new data to maintain its performance over time.

High Bias in Dataset

Bias is the error that calculates the difference between the average prediction of our model and the actual value that we are trying to predict.

A model suffering from high bias is a simple model **which pays very little attention to the training data.**

This type of model always leads to a high error on both **training and test data.**

Let's take an example.

Suppose we want our model to predict the animal by showing photos of animals.

We trained the model on only one attribute **pointing-ears.**

Then we showed the image of a cat to the model.

So the model predicted it as a fox also has pointed ears.



Ex: In Covid dataset, if the model is trained mainly on “Recovered” class, it will fail to predict outcomes for “Expired” class well.
[Too much data belonging to a single ‘Y’ type]

High Variance in Dataset

Variance is the opposite of Bias.

Variance can be defined as the model's sensitivity to fluctuations in the data, if the model is allowed to view the data too many times, it will learn very well for only that data.

It will capture most patterns in the data, but **it will also learn from the unnecessary data present, or from the noise.**

In a Classification problem, too many 'Y' outcomes
[Multiple classes]

Ex: In Covid dataset, if there were 5 outcomes, and each outcome had only 10 rows of data, it may be overfitted.
Any test value will likely be overfitted.

We need to ensure "low variance and low bias" in the training dataset



BITS Pilani
Pilani Campus



Machine Learning Generations

Three Generations of ML

Gen 1: Basic Machine Learning

Gen 2: Deep Learning [based on Artificial Neural Networks]

Gen 3: Transfer Learning / Transformers



Gen 1. Basic Machine Learning

This generation includes traditional machine learning algorithms that rely on manually engineered features and mathematical models to find patterns in data. These algorithms typically perform well on structured data and smaller datasets.

Algorithms for solving Classification problems:

- Naïve Bayes
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)
- Logistic Regression

Gen 2. Deep Learning

Deep Learning models are based on artificial neural networks with many layers (hence "deep") that automatically learn features from the data. They perform well on large datasets and can handle unstructured data such as images, text, and audio.

- Convolutional Neural Networks (CNNs) - for Image Processing & Video Analysis
- Recurrent Neural Networks (RNNs) - NLP, speech recognition, and time series analysis
- Long Short-Term Memory Networks (LSTMs) - NLP



Gen 3. Transfer Learning / Transformers

This generation leverages pre-trained models on large datasets and fine-tunes them on specific tasks. Transfer learning and transformers allow models to learn more effectively with less data and training time.

Ex: ChatGPT 4.0 or 01 models for NLP tasks like sentiment analysis, language translation, text summarization etc.

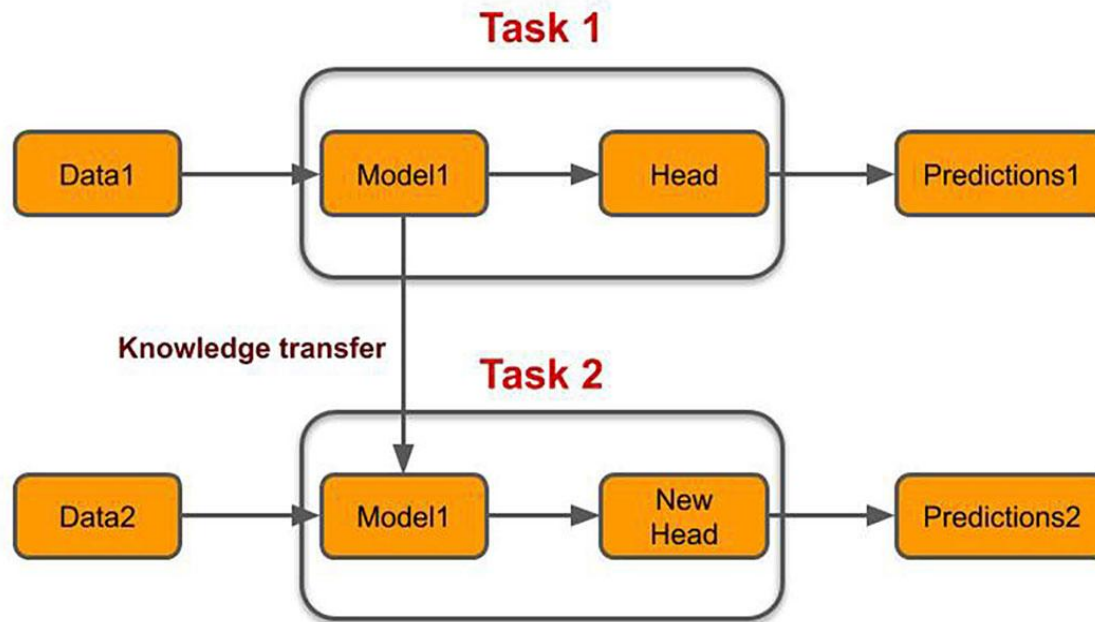
Transfer Learning

Transfer learning is a deep-learning technique where a model is first pre-trained on a data-rich task before being fine-tuned on a downstream task, has emerged as a powerful technique in natural language processing (NLP).

The effectiveness of transfer learning has given rise to a diversity of approaches, methodology, and practice.

Transfer Learning

Transfer Learning



Model developed for task 1 is reused for other tasks

Transformer Architectural Models

Sl. No	Transformer Model	Company & Info
1	BERT (Bidirectional Encoder Representations from Transformers) BERT Base - 12 encoder & 768 hidden layers BERT Large - 24 encoder & 1024 hidden layers	Google Pre-trained on unlabeled data extracted from BookCorpus with 800M words and English Wikipedia with 2500M words Machine Translation, Question Answering
2	GPT (Generative Pre-trained Transformer) GPT-2 -> Trained for 1.5B parameters for a dataset of 8 Million Web Pages GPT-3 -> Trained on 570GB of text from Textbooks and Wikipedia GPT-NEO -> Open source version of GPT-3 GPT 3.5 / 4.0 / 4o -> ChatGPT 4.0 is trained on 1.76 Trillion parameters	OpenAI Text Generation or Word prediction, question answering, summarization
3	T5 (Text-To-Text-Transfer-Transformer Model)	Question Answering, Language Translation
4	RoBERTa (Robustly Optimized BERT Pretraining Approach) Trained on 360GB of text	Facebook Multi-class Text Classification

All these models support Deep Learning Libraries TensorFlow (Google) and PyTorch (Facebook)

Machine Learning Types

REGRESSION

Predict continuous values

Linear and Non-linear
Regression

Maps input/s to continuous
output variable

Algorithm - Linear regression,
etc

CLASSIFICATION

Predicts/Classifies the
discrete values

Binary or Multi-class
Classifier

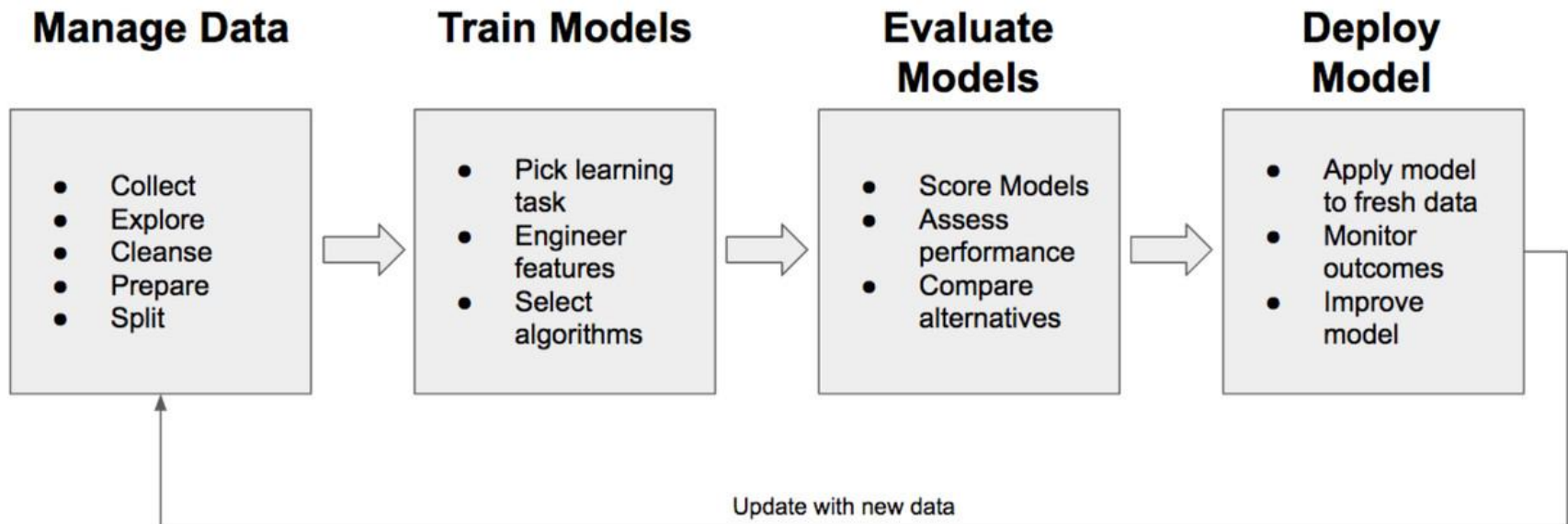
Maps input/s to discrete
output variable

Algorithms - Logistic
regression, etc

Search for Datasets and Models - <https://www.kaggle.com/>

Machine Learning Life Cycle

Machine Learning Modeling Cycle



Model Evaluation Metrics

Evaluation Metrics vary based on type of models:

Regression Models

- Mean Absolute Error (MAE)
- Root mean squared error (RMSE)
- R Squared or Adjusted R Square

Classification Models

- Confusion Matrix
- Accuracy (%)
- Precision
- Recall
- F1-score

Refer PPT - "Classification - Confusion Matrix and Metric calculations"

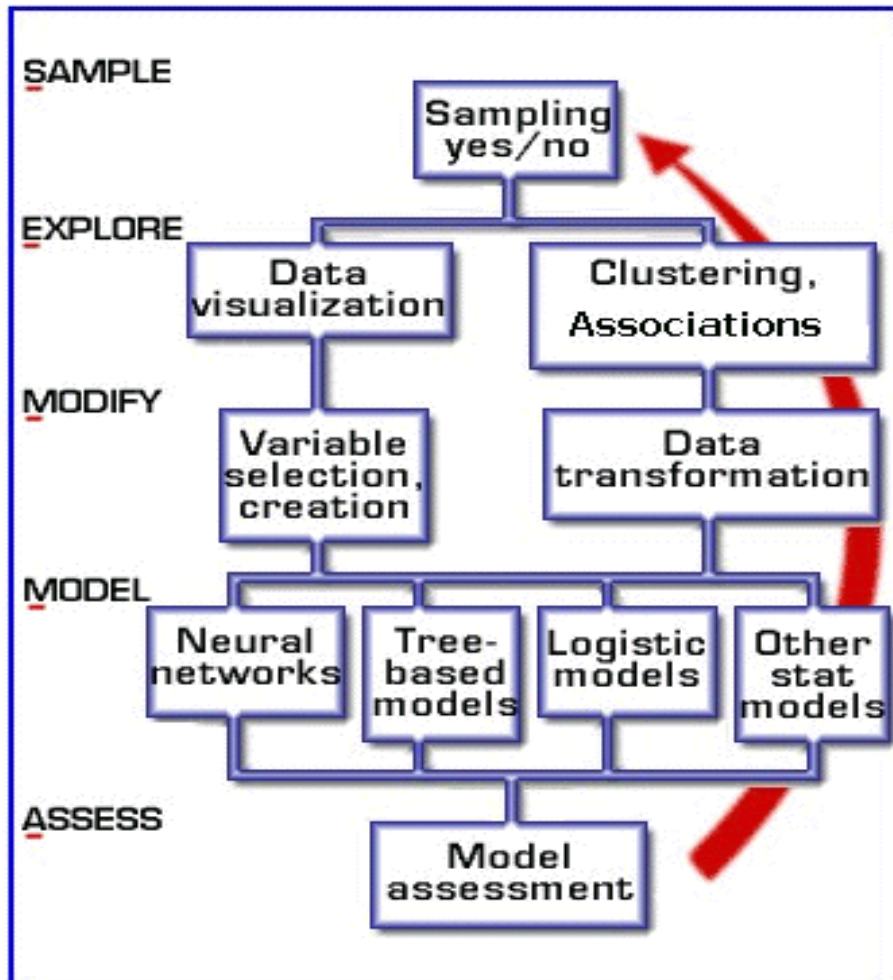


BITS Pilani
Pilani Campus



SEMMA Methodology

SEMMA



- SEMMA stands for Sample, Explore, Modify, Model, Assess
- SEMMA is focused on the model development aspects of data science, ideal for “Machine Learning” projects
- Ideal for the Data Analyst / Data Scientist roles

1. Sample

- Sampling the data by extracting a portion of a large data set big enough to contain the significant information, yet small enough to manipulate quickly.
- Partitioning the data to create training and test samples.
- Identifying dependent and independent variables influencing the process.

2. Explore

- Exploration of the data by searching for unanticipated trends and anomalies in order to gain understanding and ideas.
- Perform Univariate analysis (single variable) and multivariate analysis (relationships)

3. Modify

- Modification of the data by creating, selecting, and transforming the variables to focus the model selection process.

4. Model

- Apply variety of machine learning techniques to produce a projected model
[Basic Machine Learning, Deep Learning, Transfer Learning]

5. Assess

- Assessing the data by evaluating the usefulness and reliability of the findings from the ML process and estimate how well it performs. Deploy the selected best model.

SEMMA - Case Study



Covid Patient Discharge Prediction (Dataset: 2nd Wave April-2021 to June 2021)

Type of Project: Machine Learning

1. Sample : Dataset size: 1233 patients suffering from Covid
2. Explore: Univariate (Null values, Mean, basic statistics), Bivariate (correlation - pearson, chi square)
3. Modify : PCA (Principal Component Analysis)
4. Model : Feature Engineering, Subset selection

Final Variables:

X: Age, Gender, Co_morbid, Admit Date, Discharge date, days of stay, covid_severity

Y: Discharge Type (Recovered, Expired)

Models applied: Support Vector Machine, Naïve Bayes, Logistic Regression, Decision Trees, KNN, ANN, Random Forest

5. Assess: Best Model with highest Accuracy



BITS Pilani
Pilani Campus



MLOps

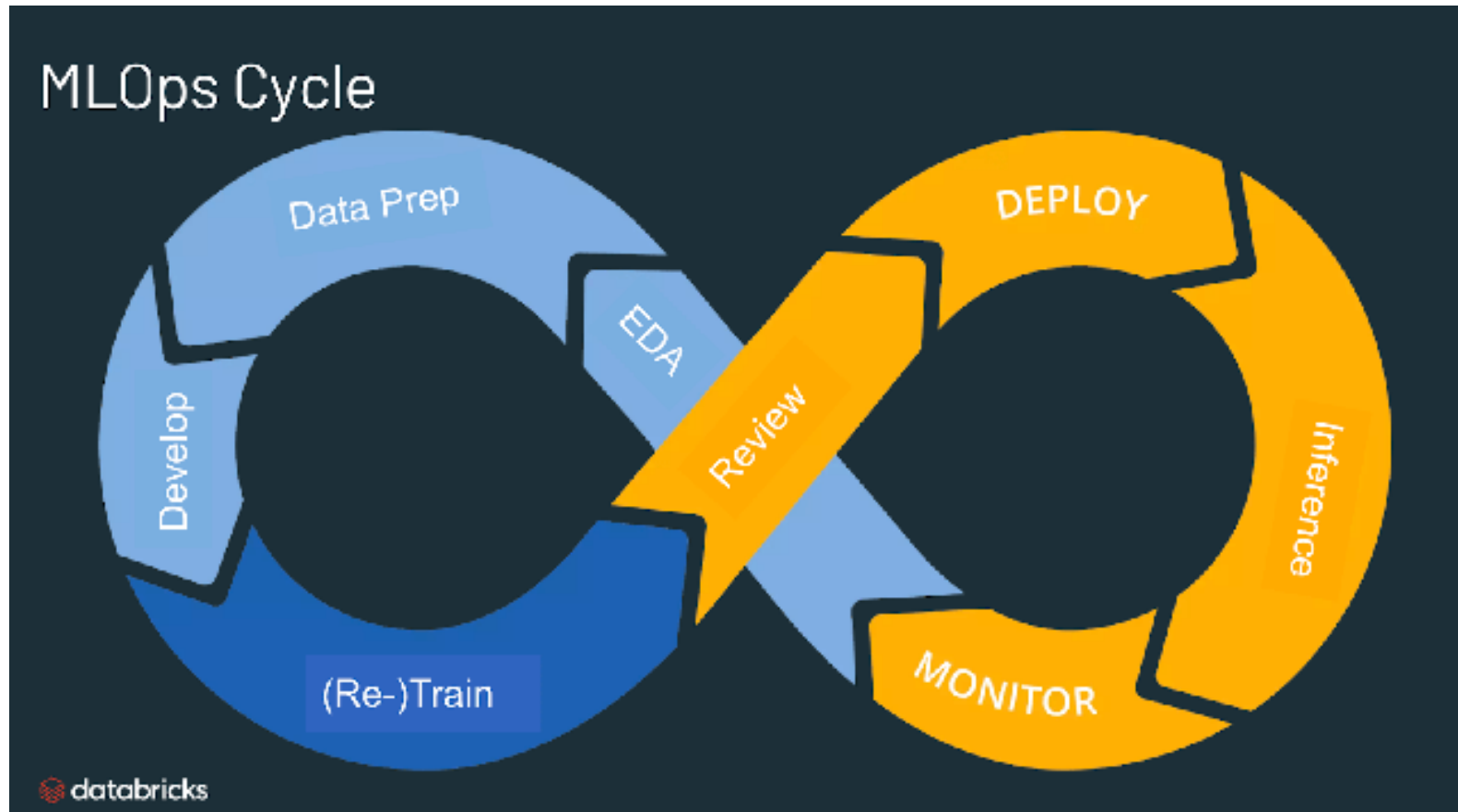


MLOps

- MLOps stands for **Machine Learning Operations**.
- A set of practices to standardize and streamline the process of developing and deploying machine learning models.
- It covers the entire machine learning workflow, including data collection, machine learning model development, training, deployment, and model management.

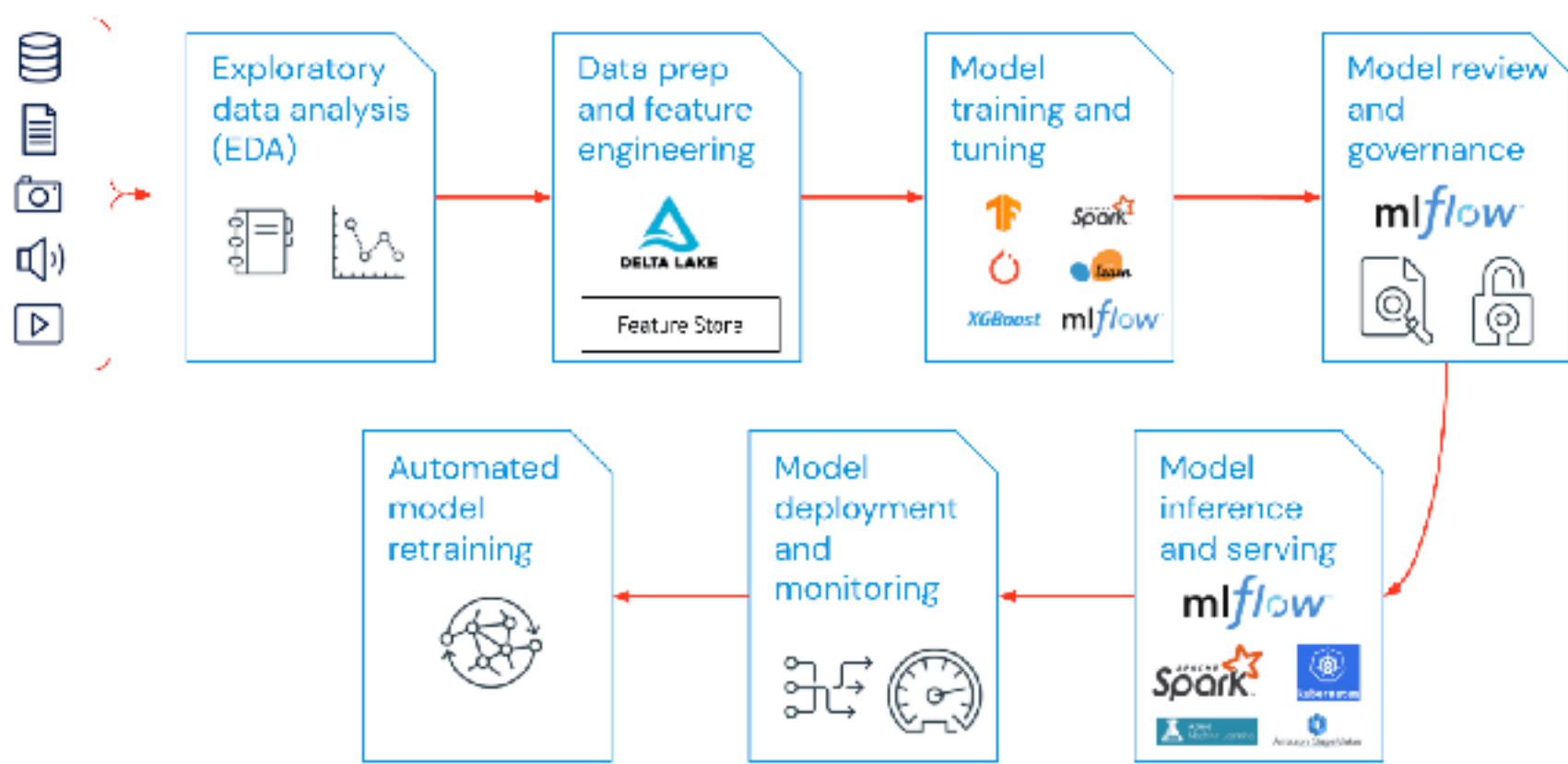
<https://www.databricks.com/glossary/mlops>

MLOps Lifecycle



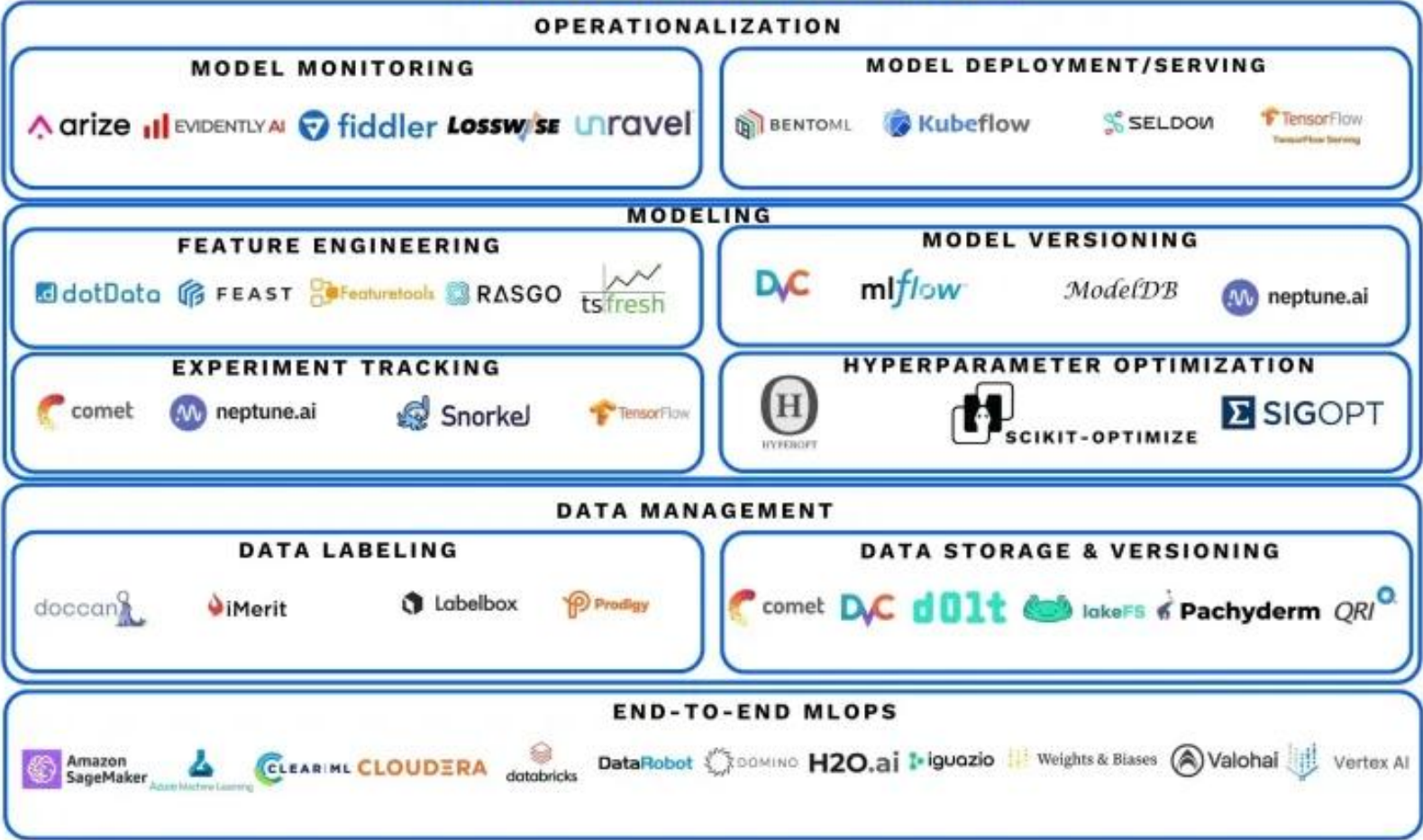
<https://www.databricks.com/glossary/mlops>

MLOps Components



<https://www.databricks.com/glossary/mlops>

MLOps Tools Landscape

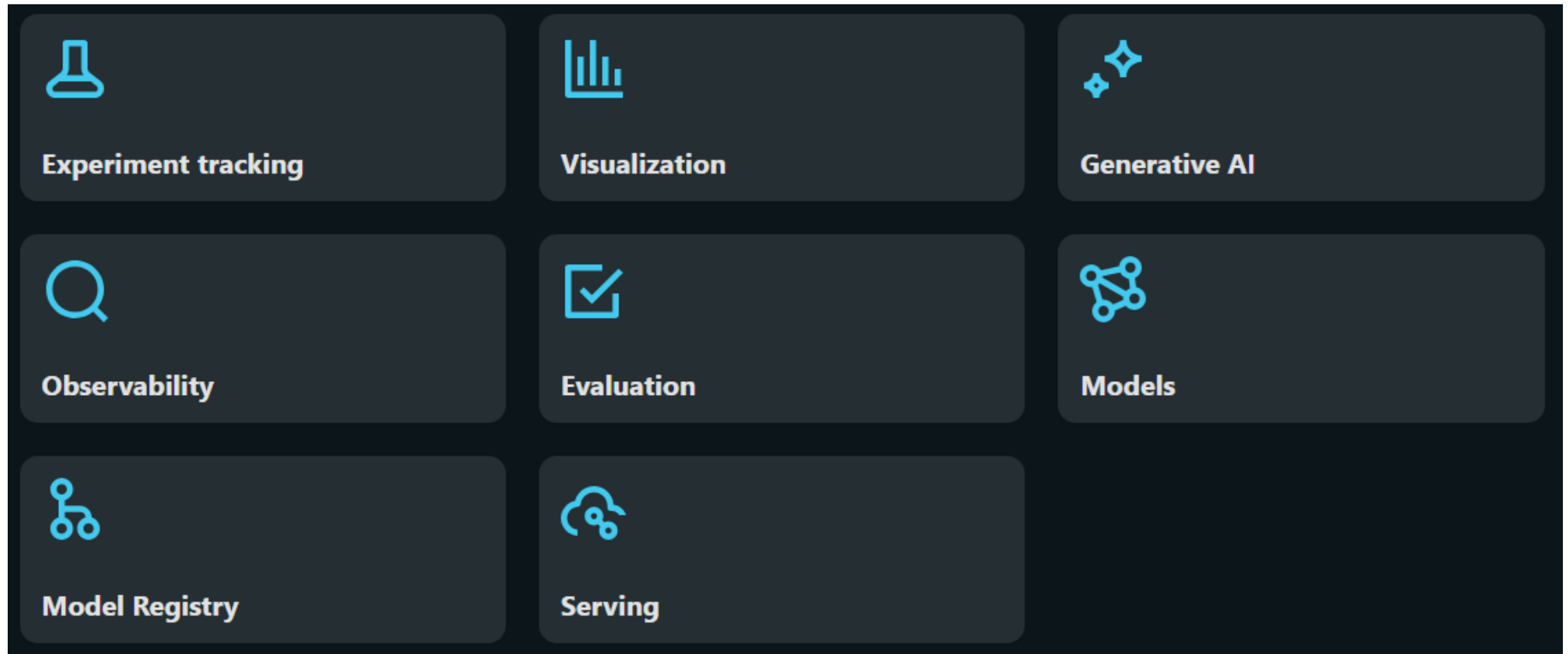


MLFlow

- MLflow is an open-source platform.
- MLflow allows us to log parameters, code versions, metrics, and artifacts for each run, making it easier to keep track of experiments and results
- Developed by Databricks, MLflow provides a suite of tools to streamline the process of developing, deploying, and monitoring machine learning models.
- It is particularly useful for organizations and teams working on multiple machine learning projects that require collaboration, reproducibility, and scalability.

Ref: <https://mlflow.org/>

MLFlow Components



Ref: <https://mlflow.org/>

Experiment Tracking

- **Purpose:** Helps track and log the details of machine learning experiments, such as model parameters, metrics, artifacts, and source code versions.
- **Features:** Provides a user-friendly interface to view and compare multiple experiments, making it easier to reproduce results and manage the progress of different ML models over time.

Visualization

- **Purpose:** Offers tools to visualize various metrics and outputs from experiments, such as accuracy, and performance metrics.
- **Features:** Allows users to create interactive plots to analyze and compare results across multiple runs, helping in identifying trends, anomalies, and the best-performing models.

Generative AI

- **Purpose:** Focuses on functionalities specific to generative AI models, such as managing experiments related to text generation, image synthesis, or other generative tasks.
- **Features:** Could include features for tracking, evaluating, and optimizing generative models, though this specific area may depend on the MLflow version or plugins in use.

Observability

- **Purpose:** Provides insights into the operational aspects of ML models, such as monitoring model performance, drift, and data changes over time.
- **Features:** Helps track models in production, ensuring they continue to perform as expected. It might include alerts for model degradation or unexpected behavior.

Evaluation

- **Purpose:** Offers tools to evaluate model performance based on different metrics.
- **Features:** Can automate the process of evaluating models against specific criteria or datasets, providing a consistent method to assess model quality and suitability for deployment.

Models

- **Purpose:** Central place to manage different versions of models.
- **Features:** Includes a model repository where users can register and store trained models, manage model versions, and track their deployment status. It enables collaboration among teams by sharing model artifacts and versions.

Model Registry

- **Purpose:** Provides a collaborative platform to manage the full lifecycle of machine learning models, from development to deployment.
- **Features:** Supports model versioning, stage transitions (such as staging to production), annotations, and approval processes. Ensures a controlled and organized way to handle model versions and their lifecycle management.

Serving

- **Purpose:** Facilitates the deployment of ML models to production environments.
- **Features:** Provides tools to deploy models as REST APIs or integrate them into existing applications or microservices, supporting real-time predictions and scalability. It helps automate the deployment process and ensures models are easily accessible for use.

MLFlow Demo for Covid Dataset

Objective

- To demonstrate a simple MLOps experiment using the “MLFlow” tool

Main (function)

- > preprocess_data
- > train_model
- > evaluate_model
- > log_to_mlflow



Metrics Summary

Accuracy (0.785): The model correctly predicted about 78.5% of the instances.

CPU Usage (11.3%): The model used 11.3% of the CPU's processing capacity during execution.

Memory Usage (85.8%): The model used 85.8% of the system's memory during execution.

Model Training Time (0.09 seconds): The model took about 0.09 seconds to train.

MLFlow - Artifacts

MLmodel: Defines the model's metadata and structure, specifying the model type, dependencies, and entry points for loading and running the model.

conda.yaml: Captures the Conda environment configuration, listing all dependencies and their versions to recreate the environment for running the model.

model.pkl: Stores the serialized (pickled) version of the trained model, allowing it to be loaded and used for predictions without retraining.

python_env.yaml: Specifies the Python environment, including the Python version and pip dependencies, ensuring the model runs consistently across different setups.

requirements.txt: Lists all Python dependencies required to run the model, serving as an alternative to conda.yaml for environments that use pip instead of Conda.

*pip is a package manager ONLY for python. Use pip if your project is python-only, and no dependencies on other languages

*conda is multi-language package manager. Resolves dependencies across languages.

Types of System Metrics in MLFlow

cpu_utilization_percentage

system_memory_usage_megabytes

system_memory_usage_percentage

gpu_utilization_percentage

gpu_memory_usage_megabytes

gpu_memory_usage_percentage

gpu_power_usage_watts

gpu_power_usage_percentage

network_receive_megabytes

network_transmit_megabytes

disk_usage_megabytes

disk_available_megabytes

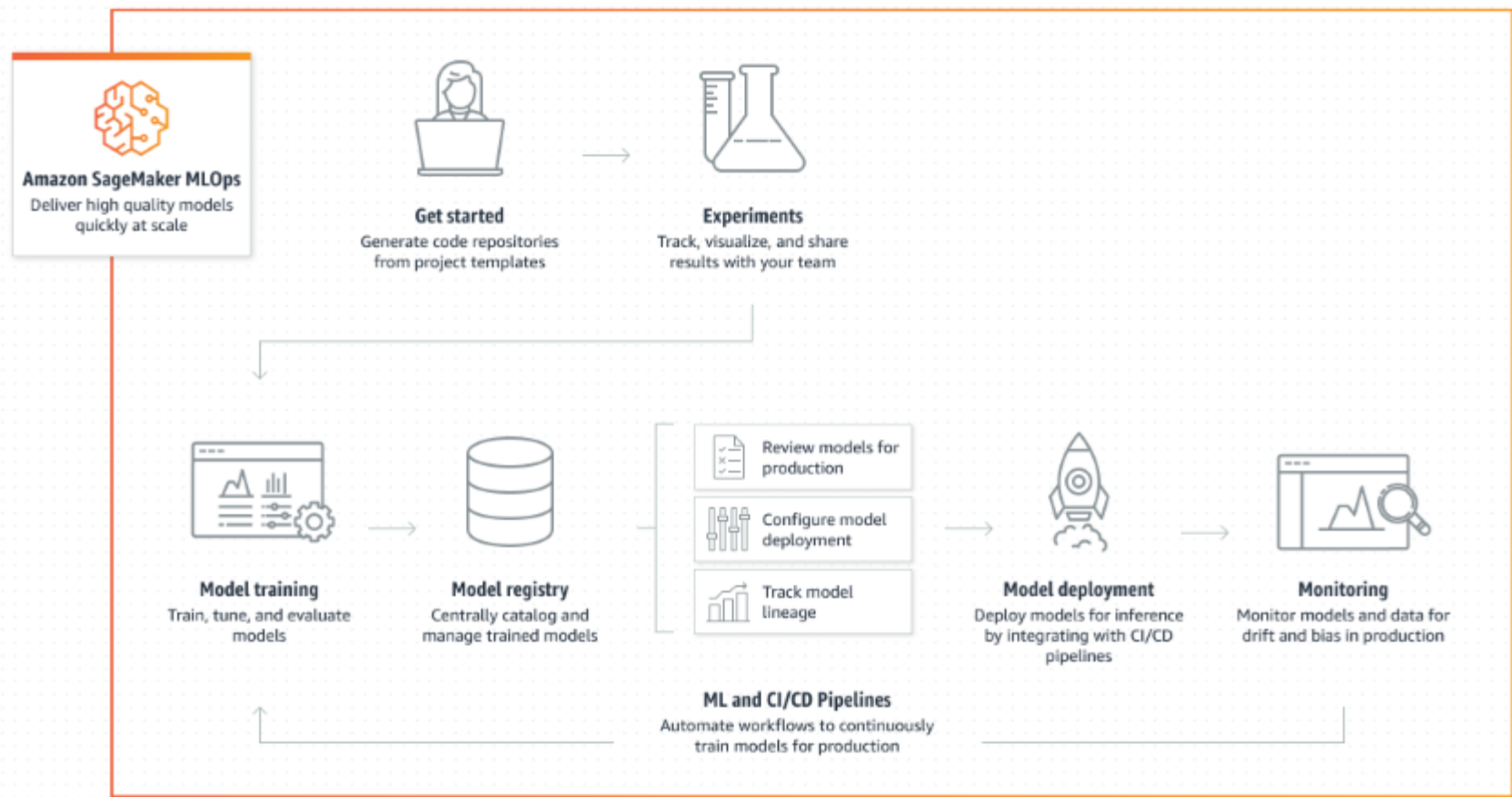
Ref - <https://mlflow.org/docs/latest/system-metrics/index.html>

MLOps Demo using SageMaker

- Amazon SageMaker is a MLOps platform
- Build, train, and deploy machine learning (ML) models for any use case with fully managed infrastructure, tools, and workflows



SageMaker



<https://aws.amazon.com/sagemaker/mlops/?sagemaker-data-wrangler-whats-new.sort-by=item.additionalFields.postDateTime&sagemaker-data-wrangler-whats-new.sort-order=desc>



Sagemaker Canvas vs Studio

- 1. SageMaker Canvas:** Low-code/no-code interface designed to make it easier for users without extensive machine learning expertise to build, train, and deploy machine learning models. It offers a visual interface where users can drag and drop components to define the data flow and model training process.
- 2. SageMaker Studio:** Is an integrated development environment (IDE) for machine learning. It provides a full-featured environment for data scientists and machine learning engineers to build, train, tune, deploy, and monitor machine learning models at scale. SageMaker Studio offers a range of tools and features, including Jupyter notebooks, experiment management, model debugging, and automated model tuning.

Sagemaker

1. **Data Wrangler**
2. **Dataset:** “Pima Indian Diabetes” Dataset
3. **Model:** AutoML model created by SageMaker [Auto ml systems automatically find, select, and optimize the best machine-learning model for a given dataset using sophisticated methods and techniques.]
4. **Model Building:** Select dataset -> Build -> Analyze -> Predict -> Deploy
[It will deploy to an endpoint, from where predictions can be made]
5. **SageMaker Model Registry** - SageMaker Model Registry can be used to catalog and manage different model versions.

Demo of “Quick Build AutoML Model” for Pima Indian Diabetes Dataset



Thank You!